

Hybrid Recommender Systems Analysis

This report explains the implementation of hybrid recommenders to combine collaborative and content-based signals. We explored two different strategies: Weighted blending (from previous experiments) and candidate generation + reranking - cascade hybrid

1. The Weighted Blending Attempt

In our earlier similarity-based models experiment, we implemented a simple weighted blending hybrid: $\text{Score} = 0.8 * \text{CF_Score} + 0.2 * \text{CB_Score}$

It slightly underperformed pure Collaborative Filtering at the very top (K=10):

- Pure CF (Cosine): NDCG@10 = 0.1620
- Weighted Hybrid: NDCG@10 = 0.1598

Why it failed:

MovieLens is a very dense dataset. The user interaction patterns (CF) are much stronger than simple movie metadata like genres or release year (CB). By naively mixing these scores globally for all items, the content-based signal essentially added noise to the highly accurate collaborative predictions for users who already had a rich history

This led us to design a smarter hybrid structure

2. Candidate Generation + Reranking

Instead of blindly mixing scores, we built a two-stage cascade hybrid (implemented in `src/models/hybrid.py`)

How it works:

1. The collaborative filtering model (Item-Item Cosine) quickly scans the entire catalog and retrieves the top 100 relevant movies. This step ensures high recall based on crowd behavior
2. The content-based model (TF-IDF on titles and genres) scores *only* these 100 candidates. It re-sorts them to match the user's specific semantic preferences

We chose this structure because it restricts the noise of the content-based model. By applying the CB model only to the top 100 items, we guarantee that any movie recommended is already considered "good" by the crowd (CF). The reranker just fine-tunes the final top-10 list based on the user's current mood or specific genre tastes. This is also the standard architecture used in large-scale industry systems because applying a heavy model to millions of items is too slow.

3. Experiment Results

We evaluated the Cascade Pipeline on a subset of users using a temporal split

Model	NDCG@10	Precision@10	Recall@10
CF (Cosine)	0.1670	0.1520	0.0928
Cascade Hybrid (CF -> CB)	0.0891	0.0855	0.0530
CB (TF-IDF)	0.0236	0.0195	0.0132

4. Who benefits from it?

If pure CF has higher average metrics, why build this hybrid? Because offline metrics on dense datasets hide specific use cases. The Cascade Hybrid specifically benefits:

1. Users with specific niche tastes:

If the CF model retrieves a mix of Action, Romance, and Sci-Fi based on general co-viewing patterns, but the user strictly loves Sci-Fi, the CB reranker will correctly push the Sci-Fi movies to the top 10

2. The cold-start scenario:

Users with very few ratings confuse pure CF models. In a real system, we wouldn't use this hybrid for power users. We would use it as a fallback strategy for new users, where the collaborative signal is too weak to be trusted on its own

3. Tail items:

Niche movies that barely make it to rank #80 or #90 in the CF generation phase (because they lack mass popularity) can be pushed to the top 3 by the reranker if they perfectly match the user's content profile. This breaks the "rich-get-richer" popularity bias of pure CF